

基于 ucloud 云主机和阿里云镜像仓库实现 GitHub 的 CI/CD

GitHub Actions是GitHub的一个持续集成和持续交付（CI/CD）的平台，可以自动化构建、测试和部署Pipelines。

Workflow（工作流）

Workflow其实就是一个可配置的自动化过程，会运行一个或多个Job。Workflow定义在仓库的 .github/workflows 文件夹的一个YML或者YAML文件中，并在仓库的Event触发时运行，也可以配置成手动触发运行，或者定时触发。一个Workflow中甚至可以引用另一个Workflow。

可以简单理解，一个YML配置文件就是一个Workflow，仓库可以在.github/workflows文件夹下拥有多个YML文件，即拥有多个Workflows。每个Workflow可以执行一组不同的步骤。例如，一个仓库中有两个Workflows，就可以有一个Workflow用来构建和测试PR（Pull Request），另一个Workflow用来每次发布Release版本时自动部署应用。

Event（事件）

Event是仓库中的一个用来触发Workflow运行的特殊活动，有不同类型，如某人创建了一个PR、新建了一个Issue等，这些都是可以用来触发Workflow运行的Event。

Job（工作）

Job是Workflow中的一组步骤，在同一个Runner（运行者）上运行。每个步骤要么是将执行的Shell脚本，要么是将运行的Action。各步骤是按顺序执行的，并且相互依赖。由于各步骤是在同一个Runner上运行，因此可以将数据从一个步骤共享到另一个步骤。可以配置Job与其他Job的依赖关系；默认情况下，Job之间没有依赖关系，并且彼此并行运行。当一个Job依赖于另一个Job时，它将等待从属Job完成，然后才能运行。

Action

Action是GitHub Actions平台的自定义应用程序，用于执行复杂但经常重复的任务。Action可以减少在Workflow的YML文件中编写的重复代码。Action可以从GitHub上拉取仓库代码，为构建环境设置正确的工具链。我们可以编写自己的Action，也可以在GitHub Marketplace中寻找适合使用的Action。

Runner（运行者）

Runner是在Workflow被触发时运行它们的服务器。每个Runner一次可以运行一个Job。GitHub提供了Ubuntu Linux、Windows和macOS Runner来运行Workflow。每个Workflow都在全新的预先配置好的虚拟机中执行。如果我们需要不同的操作系统或者特定的硬件配置，可以用自己托管的Runner来代替GitHub提供的Runner。

实验目的和实验环境

【实验目的】

- (1) 了解 GitHub Actions。
- (2) 初步了解 Workflow 语法并自行构建。
- (3) 实现 Spring Cloud 项目全自动化打包部署。
- (4) 熟悉整个 CI/CD 的原理。

【实验环境】

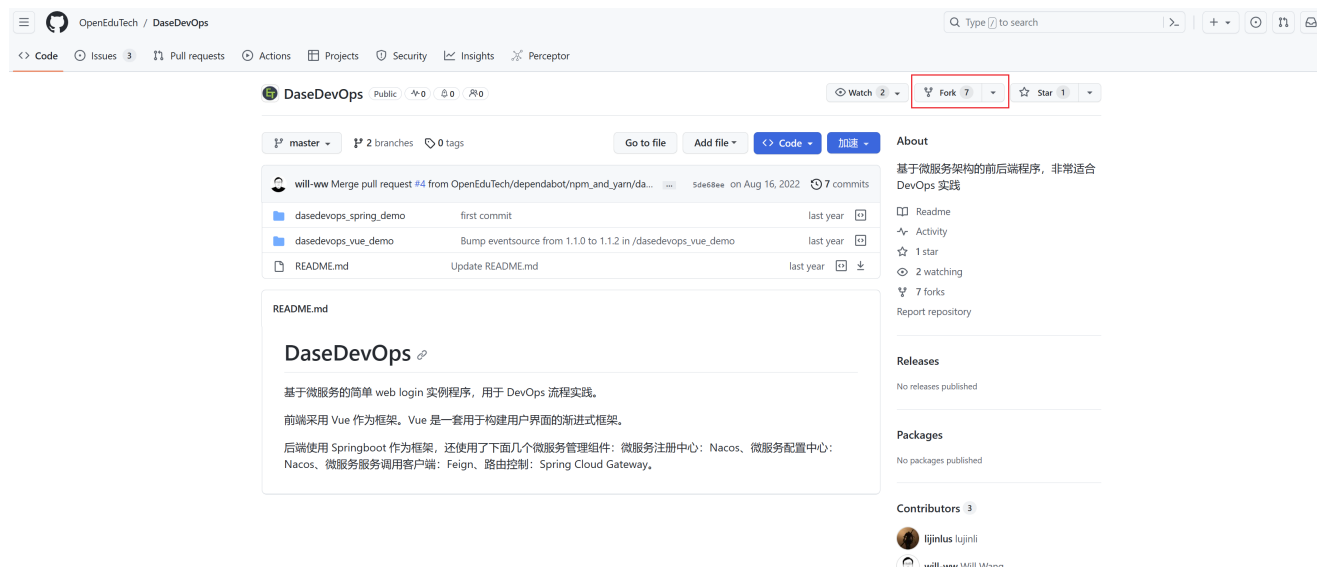
- (1) CentOS: 7.9 及以上
- (2) Docker: 20.10.7 版。
- (3) docker-compose: 1.29.2 版, build 5becea4c。
- (4) maven 3.9.5
- (5) java openjdk-1.8.0

(6) 本次实验建议使用 vscode 连接云主机, 这样就可以直接在云主机上编辑, 而不用总是上传文件。(这个可以自行上网学习, 很方便, 参考文档 https://blog.csdn.net/m0_49448331/article/details/126030161)

实验步骤

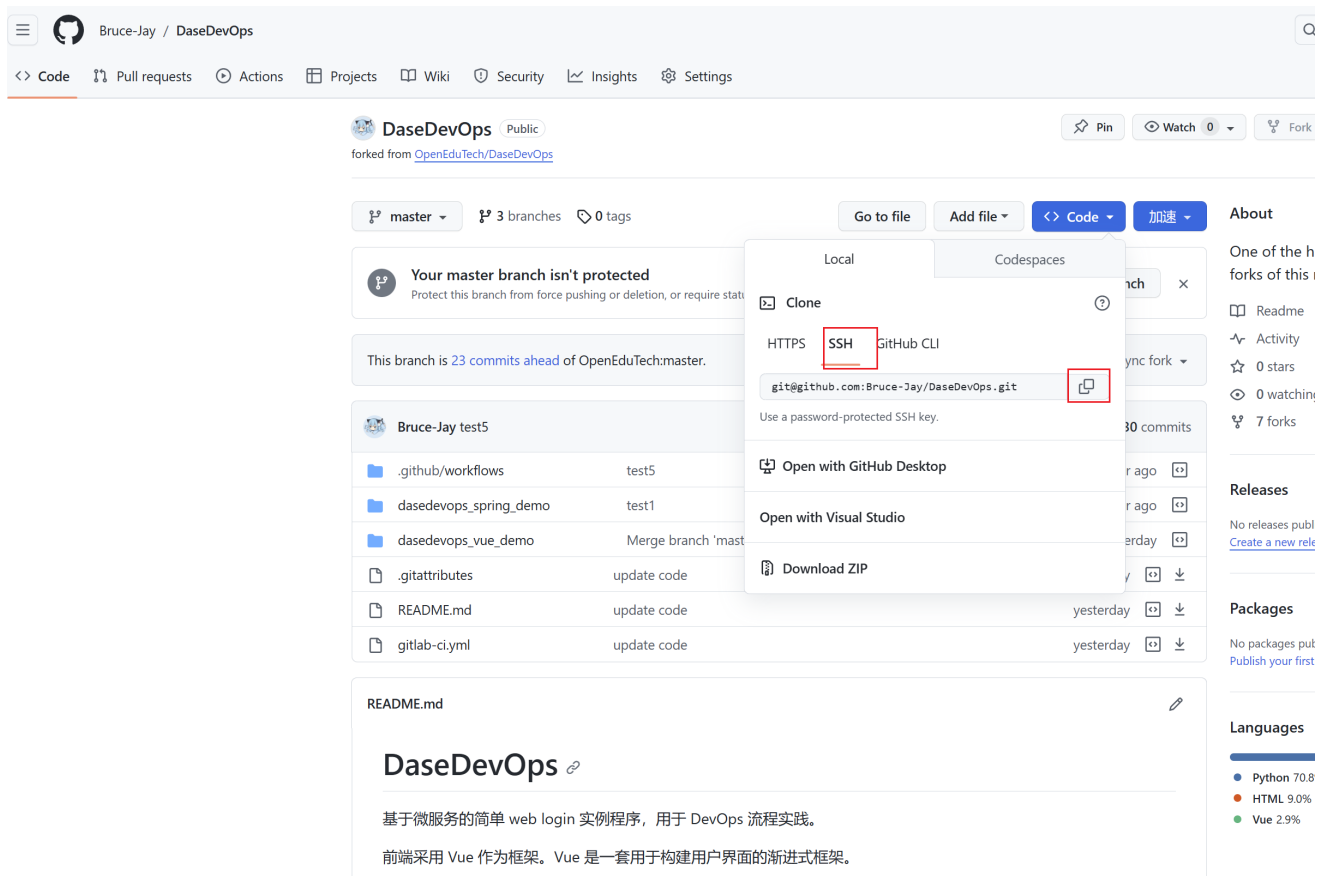
1. 复制项目代码

将 <https://github.com/OpenEduTech/DaseDevOps/tree/master> 复制到自己的账户下后, 会生成DaseDevops仓库。注意: 下面所有图中显示的devops_demo仓库其实对应的是DaseDevops仓库下的dasedevops_spring_demo项目。



随后, 在自己的云主机任意目录下 (我这里是 /root) 执行 git clone 克隆仓库。使用 ssh 克隆。在克隆之前, 可能会报没有 git 命令的错误, 需要你手动安装一下 `yum install -y git`

请重点检查: 是 fork 后的仓库, 而不是 OpenEduTech 的仓库!!! 不然会无法 push



在云主机上，生成本机密钥，用于免密认证。引号内的部分需要更换成你自己 github 绑定的邮箱。

```
cd .ssh
ssh-keygen -t rsa -C "102155014xx@stu.ecnu.edu.cn"
```

一路回车，待成功生成密钥后，执行命令 `cat id_rsa.pub` 将打印出的公钥复制到 github 中。

在 github 上配置云主机的 ssh key，点击头像，

SSH keys

New SSH key

This is a list of SSH keys associated with your account. Remove any keys that you do not recognize.

Authentication Keys

	EasonLinBlog SHA256: IiPQ3xUwZ1IgLT0ys33v9s1aKrmLw+9MLi3gb45sMo Added on May 8, 2022 Last used within the last 4 months — Read/write	Delete
	Mac ssh key SHA256: XoApH6W+tLkhL0D4+5eGtLZH421QDcN3GugIQPR+LGQ Added on Jun 16, 2023 Last used within the last 4 months — Read/write	Delete
	Panasonic Key SHA256: Prmd/ZewsruRcmF3rmcs/r0kvB5P9+mTeXgWsvhPqU Added on Aug 21, 2023 Last used within the last 4 weeks — Read/write	Delete
	MechRev SHA256: aTqpnMQEtFAYKsZd+592Y94Aoo4xUkM/vp1Nr1V1zw Added on Sep 27, 2023 Last used within the last week — Read/write	Delete
	aws SHA256: vntZmErOm1C9QtuHEmoUiR0LwI7KNFFTMESVDSJCoE Added on Oct 6, 2023 Last used within the last 2 months — Read/write	Delete
	action SHA256: +d1ddl/doNngGv/ebp1mN31M7G0B8atf6I50yH/WALs Added on Nov 11, 2023 Last used within the last week — Read/write	Delete

密钥 Title 设置为 action，内容复制进去，这样就可以直接在云主机上免密 git push 了。

2. 建立服务器A环境

这里使用的是GitHub托管的Runner，因此服务器A的环境其实已经由GitHub处理好了，不需要我们做任何操作。如果使用自托管的Runner，感兴趣的同学可以参考相关网页。

3. 建立服务器B环境

首先，创建一个 centOS 云主机，1 核 2G 配置，20M 带宽流量计费，数据盘设置为 50 G

安装Docker(参照https://gitea.shuishan.net.cn/xslu_dase_ecnu_edu_cn/cloud-computing-course/src/branch/master/Assignment2.md),

安装docker-compose。

docker-compose 安装过程：

通过 curl 下载 docker compose

```
sudo curl -L "https://github.com/docker/compose/releases/download/1.29.2/docker-compose-$(uname -s)-$(uname -m)" -o /usr/local/bin/docker-compose
```

添加可执行权限

```
sudo chmod +x /usr/local/bin/docker-compose
```

将文件 copy 到 /usr/bin/ 目录下

```
sudo ln -s /usr/local/bin/docker-compose /usr/bin/docker-compose
```

查看版本

```
docker-compose --version
```

显示以下信息说明安装成功。

```
compose
[root@10-9-22-43 ~]# docker-compose --version
docker-compose version 1.29.2, build 5becea4c
```

安装 jdk

请注意，本次实验仅支持手动安装，不支持使用 yum 安装！！

浏览器访问 oracle 官网 <https://www.oracle.com/java/technologies/downloads/#license-lightbox> 下拉到 Java SE8，下载 x64 版本的压缩包

Java 8 Java 8 Enterprise Performance Pack Java 11

Java SE Development Kit 8u391

Java SE subscribers will receive JDK 8 updates until at least **December 2030**.

[Manual update required for some Java 8 users on macOS.](#)

The Oracle JDK 8 license changed in April 2019

The [Oracle Technology Network License Agreement for Oracle Java SE](#) is substantially different from prior Oracle JDK 8 licenses. This license permits certain uses, such as personal use and development use, at no cost -- but other uses authorized under prior Oracle JDK licenses may no longer be available. Please review the terms carefully before downloading and using this product. FAQs are available [here](#).

Commercial license and support are available for a low cost with [Java SE Universal Subscription](#).

JDK 8 software is licensed under the [Oracle Technology Network License Agreement for Oracle Java SE](#).

Java SE 8u391 checksums and OL 8 GPG Keys for RPMs

Linux macOS Solaris Windows

Product/file description	File size	Download
ARM64 RPM Package	71.06 MB	 jdk-8u391-linux-aarch64.rpm
ARM64 Compressed Archive	71.23 MB	 jdk-8u391-linux-aarch64.tar.gz
x86 RPM Package	140.62 MB	 jdk-8u391-linux-i586.rpm
x86 Compressed Archive	138.69 MB	 jdk-8u391-linux-i586.tar.gz
x64 RPM Package	137.36 MB	 jdk-8u391-linux-x64.rpm
x64 Compressed Archive	135.33 MB	 jdk-8u391-linux-x64.tar.gz

在云主机的 /usr 目录下创建 java/ 文件夹

```
cd /usr
mkdir java
cd java
```

把下载好的文件上传到 /usr/java 下

因为文件有 100M 大小，建议等待一段时间，等上传完成以后进行后续操作。

解压

```
tar -zxvf jdk-8u391-linux-x64.tar.gz
```

解压以后，配置环境变量

```
vi /etc/profile
```

在文件末尾添加

```
export JAVA_HOME=/usr/java/jdk1.8.0_391
export PATH=$JAVA_HOME/bin:$PATH
export CLASSPATH=.:$JAVA_HOME/lib/dt.jar:$JAVA_HOME/lib
```

更新环境变量

```
source /etc/profile
```

输入 java -version 应该可以看到安装成功的消息。

```
● [root@10-9-22-43 DaseDevOps]# java -version
java version "1.8.0_391"
Java(TM) SE Runtime Environment (build 1.8.0_391-b13)
Java HotSpot(TM) 64-Bit Server VM (build 25.391-b13, mixed mode)
```

安装 maven

浏览器访问 maven 的下载页面 <https://maven.apache.org/download.cgi>

点击链接下载

Downloading Apache Maven 3.9.5

Apache Maven 3.9.5 is the latest release: it is the recommended version for all users.

System Requirements

Java Development Kit (JDK)	Maven 3.9+ requires JDK 8 or above to execute. It still allows you to build against 1.3 and other JDK versions by using toolchains .
Memory	No minimum requirement
Disk	Approximately 10MB is required for the Maven installation itself. In addition to that, disk space will be used for your local Maven repository (default 500MB).
Operating System	No minimum requirement. Start up scripts are included as shell scripts (tested on many Unix flavors) and Windows batch files.

Files

Maven is distributed in several formats for your convenience. Simply pick a ready-made binary distribution archive and follow the [installation instructions](#). Use a checksum to verify the integrity of the download. In order to guard against corrupted downloads/installations, it is highly recommended to [verify the signature](#) of the release bundles against the public [KEYS](#) using the [gpg](#) command.

	Link	Checksums
Binary tar.gz archive	apache-maven-3.9.5-bin.tar.gz	apache-maven-3.9.5-bin.tar.gz.sha512
Binary zip archive	apache-maven-3.9.5-bin.zip	apache-maven-3.9.5-bin.zip.sha512
Source tar.gz archive	apache-maven-3.9.5-src.tar.gz	apache-maven-3.9.5-src.tar.gz.sha512
Source zip archive	apache-maven-3.9.5-src.zip	apache-maven-3.9.5-src.zip.sha512

- 3.9.5 [Release Notes](#) and [Release Reference Documentation](#)
- [latest source code from source repository](#)
- Distributed under the [Apache License, version 2.0](#)

之后上传到 ucloud 云主机的 /usr/local 目录下，进入 /usr/local 目录，解压

```
tar -xvf apache-maven-3.9.5-bin.tar.gz
```

执行 `vi /etc/profile` 配置环境变量

```
MAVEN_HOME=/usr/local/apache-maven-3.9.5
export PATH=${MAVEN_HOME}/bin:${PATH}
```

重载环境变量

```
source /etc/profile
```

查看是否安装成功，输入 `mvn -v` 命令，显示相关信息

```
● [root@10-9-22-43 DaseDevOps]# mvn -v
Apache Maven 3.9.5 (57804ffe001d7215b5e7bcb531cf83df38f93546) 版本
Maven home: /usr/local/apache-maven-3.9.5 maven 安装路径
Java version: 1.8.0_391, vendor: Oracle Corporation, runtime: /usr/java/jdk1.8.0_391/jre java
Default locale: en_US, platform encoding: UTF-8
OS name: "linux", version: "4.18.0-305.3.1.el8.x86_64", arch: "amd64", family: "unix"
```

由于 maven 官方源在国内速度比较慢，我们需要配置阿里 maven 源。cd 到 maven 的安装目录下

修改 conf 文件夹下的 settings.xml

```
sudo vi conf/settings.xml
```

修改 mirrors 下没有被注释的那一部分

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

    <id>siteServer</id>
    <privateKey>/path/to/private/key</privateKey>
    <passphrase>optional; leave empty if not used.</passphrase>
  </server>
-->
</servers>

<!-- mirrors
| This is a list of mirrors to be used in downloading artifacts from remote repositories.
|
| It works like this: a POM may declare a repository to use in resolving certain artifacts.
| However, this repository may have problems with heavy traffic at times, so people have mirrored
| it to several places.
|
| That repository definition will have a unique id, so we can create a mirror reference for that
| repository, to be used as an alternate download site. The mirror site will be the preferred
| server for that repository.
|-->
<mirrors>
  <!-- mirror
  | Specifies a repository mirror site to use instead of a given repository. The repository that
  | this mirror serves has an ID that matches the mirrorOf element of this mirror. IDs are used
  | for inheritance and direct lookup purposes, and must be unique across the set of mirrors.
  |
  <mirror>
    <id>mirrorId</id>
    <mirrorOf>repositoryId</mirrorOf>
    <name>Human Readable Name for this Mirror.</name>
    <url>http://my.repository.com/repo/path</url>
  </mirror>
  -->
  <mirror>
    <id>alimaven</id>
    <mirrorOf>central</mirrorOf>
    <name>aliyun maven</name>
    your Tomcat instance is installed, you can provide a variable here such that the variable is
    dereferenced during the build process to configure the cactus plugin.

    As noted above, profiles can be activated in a variety of ways. One way - the activeProfiles
    section of this document (settings.xml) - will be discussed later. Another way essentially
    relies on the detection of a property, either matching a particular value for the property,
    or merely testing its existence. Profiles can also be activated by JDK version prefix, where a
    value of '1.4' might activate a profile when the build is executed on a JDK version of '1.4.2_07'.
    Finally, the list of active profiles can be specified directly from the command line.

    NOTE: For profiles defined in the settings.xml, you are restricted to specifying only artifact
    repositories, plugin repositories, and free-form properties to be used as configuration
    variables for plugins in the POM.
  -->
</profiles>
<!-- profile
```

我这边是已经修改过的，同学们需要根据以下代码修改原有的代码

```
<mirror>
  <id>alimaven</id>
  <mirrorOf>central</mirrorOf>
  <name>aliyun maven</name>
  <url>http://maven.aliyun.com/nexus/content/groups/public/</url>
</mirror>
```

需要把原有的 `<block>>true</block>` 去掉

4. 建立 ucloud 阿里云容器镜像服务

本来是想通过 ucloud 使用容器镜像服务的，但是奈何 ucloud 镜像仓库无法通过 github action 的 workflow 进行登录并且操作。无奈之下只能使用阿里云的 [aliyun/acr-login](https://www.aliyun.com/acr-login) 进行 yml 文件登录到 docker 的操作。

为什么不能通过 ucloud 创建容器镜像服务然后再通过登录阿里云镜像仓库然后把镜像推送到 ucloud 中呢？不行。就像 github 中无法直接把自己的代码推送到别人的仓库中一样。

所以为了完成本次实验，需要麻烦大家再在阿里云上创建一个容器镜像服务。

首先，访问 <https://www.aliyun.com/> 点击右上角的“注册/登录”按钮，完成注册与登录操作

可以直接使用支付宝扫码登录

登录成功后，访问实例列表 <https://cr.console.aliyun.com/cn-shanghai/instances> 点击创建个人实例。

需要通过 RAM 授权，按照流程点击确认即可。

实例列表

创建企业版实例

全部展示

企业版

企业级安全托管和高效分发服务，限时优惠购买

+

什么是企业版实例？
如何无缝从自建迁移企业版实例？

个人实例

个人容器镜像托管服务，限额使用

+

什么是个人版实例？

提示

ACR 面向个人开发者提供了 ACR 个人版，面向安全及分发性能需求高的企业客户提供了 ACR 企业版，查看文档了解更多。

一级功能	二级功能	容器镜像服务个人版	容器镜像服务企业版
制品管理	制品托管类型	容器镜像	容器镜像、Helm Chart 等 OCI 制品
	制品自动清理	-	✓
	容器镜像构建服务	✓	✓ (企业级增强)
	自定义域名	-	✓
制品分发	SLA 保障	-	✓
	全球同步加速	-	✓
	大规模分发加速	-	✓

创建个人版

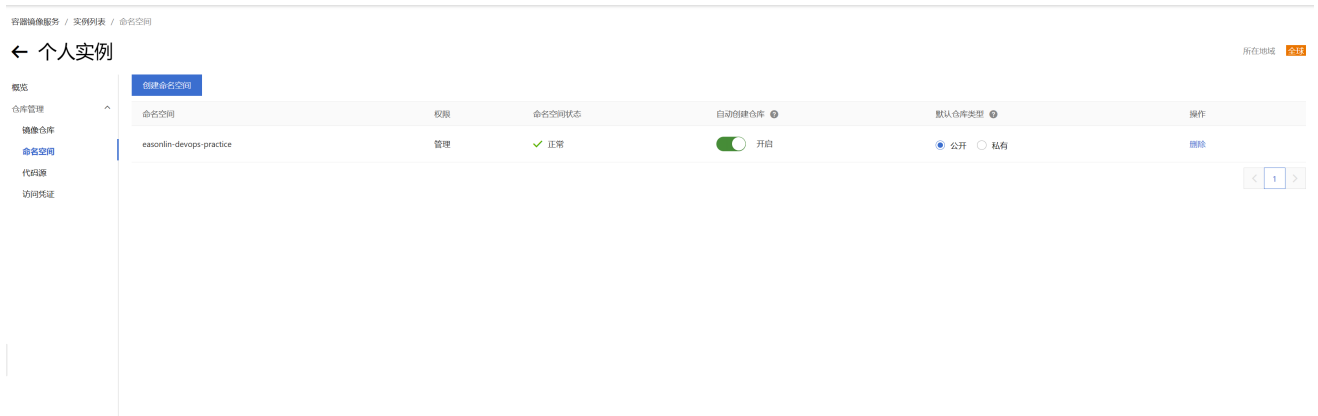
创建企业版实例

取消

创建完个人版实例以后，需要设置 registry 登录密码。

然后点击 命名空间 -> 创建命名空间 -> 给自己的命名空间起一个名字吧

推荐使用自己的姓名拼音之间加 "-" 号进行命名。如果出现重名，就再加一些符号。



需要把 默认仓库类型 设置为公开。

作业 1： 创建完成以后，进入个人实例-命名空间页面，截图插入实验报告。

5.接下来，编写本项目的 workflow 文件。在项目根目录下新建 .github/workflows 文件夹将以下代码保存在 docker-publish.yml

```
name: SpringCloud CI/CD with Docker
on:
  push:
    branches: [ master ]
  pull_request:
    branches: [ master ]

env:
  MODULE_1: gateway
  MODULE_2: hello
  MODULE_3: login
  MODULE_4: provider_one
  MODULE_5: provider_two
  MODULE_6: provider_three

jobs:
  build:
```

```

runs-on: ubuntu-latest
steps:
- uses: actions/checkout@v2
- name: Set up JDK 8
  uses: actions/setup-java@v2
  with:
    distribution: 'temurin'
    java-version: '8'
    cache: 'maven'
- name: Build with Maven
  run: |
    whoami
    pwd
    cd dasedevops_spring_demo
    mvn -B package --file pom.xml
    pwd
- name: Login to Aliyun Container Registry (ACR)
  uses: aliyun/acr-login@v1
  with:
    login-server: https://registry.cn-shanghai.aliyuncs.com
    region-id: cn-shanghai # 3
    username: "${{ secrets.ACR_USERNAME }}"
    password: "${{ secrets.ACR_PASSWORD }}"
- name: Build and push image
  run: |
    docker build -t ${ secrets.ALI_NAMESPACE_URL }/$MODULE_1:latest
./dasedevops_spring_demo/$MODULE_1
    docker push ${ secrets.ALI_NAMESPACE_URL }/$MODULE_1:latest
    docker build -t ${ secrets.ALI_NAMESPACE_URL }/$MODULE_2:latest
./dasedevops_spring_demo/$MODULE_2
    docker push ${ secrets.ALI_NAMESPACE_URL }/$MODULE_2:latest
    docker build -t ${ secrets.ALI_NAMESPACE_URL }/$MODULE_3:latest
./dasedevops_spring_demo/$MODULE_3
    docker push ${ secrets.ALI_NAMESPACE_URL }/$MODULE_3:latest
    docker build -t ${ secrets.ALI_NAMESPACE_URL }/$MODULE_4:latest
./dasedevops_spring_demo/$MODULE_4
    docker push ${ secrets.ALI_NAMESPACE_URL }/$MODULE_4:latest
    docker build -t ${ secrets.ALI_NAMESPACE_URL }/$MODULE_5:latest
./dasedevops_spring_demo/$MODULE_5
    docker push ${ secrets.ALI_NAMESPACE_URL }/$MODULE_5:latest
    docker build -t ${ secrets.ALI_NAMESPACE_URL }/$MODULE_6:latest
./dasedevops_spring_demo/$MODULE_6
    docker push ${ secrets.ALI_NAMESPACE_URL }/$MODULE_6:latest

- name: Copy single file to remote
  uses: garygrossgarten/github-action-scp@release
  with:
    local: ./dasedevops_spring_demo/docker-compose.yml
    remote: scp/devops_demo/docker-compose.yml
    host: ${ secrets.HOST }
    username: ${ secrets.USER_NAME }
    password: ${ secrets.USER_PASSWORD }
    port: ${ secrets.PORT }

```

```
depoly:
  needs: [ build ]
  name: Docker Pull And Docker-compose Run
  runs-on: ubuntu-latest
  steps:
    - name: executing remote ssh commands using password
      uses: appleboy/ssh-action@master
      with:
        host: ${ secrets.HOST }
        username: ${ secrets.USER_NAME }
        password: ${ secrets.USER_PASSWORD }
        port: ${ secrets.PORT }
        script: |
          docker-compose -f scp/devops_demo/docker-compose.yml up -d
          docker-compose -f scp/devops_demo/docker-compose.yml stop
          docker-compose -f scp/devops_demo/docker-compose.yml pull
          docker-compose -f scp/devops_demo/docker-compose.yml up -d
          docker image prune -f
```

相关 secrets 解释：

为什么需要用到 github secrets? 我们都知道，很多情况下，项目有一些隐秘信息，不能直接配置在项目内，包括但不限于: github token, 各种账号的用户名密码，私钥信息，各种网站的 api key, app key, secret key 等等。这时候就需要用到 secrets。所以根据 secrets 的特性，一经设置就不可查看！所以设置 secrets 时务必仔细确认！

secrets.ALI_NAMESPACE_URL: 对应阿里云容器镜像服务的“所在地域 + 命名空间”，我这里是“registry.cn-shanghai.aliyuncs.com/easonlin-devops-practice”，前面的 registry.cn-shanghai.aliyuncs.com 是地域，后面的 easonlin-devops-practice 是命名空间。地域和命名空间很重要，在我们登录阿里云 registry 和拉取、推送镜像时都会用到。

[操作指南](#) 制品描述

1. 登录阿里云 Docker Registry

```
$ docker login --username=tb1020550141 registry.cn-shanghai.aliyuncs.com
```

用于登录的用户名为阿里云账号全名，密码为开通服务时设置的密码。

您可以在访问凭证页面修改凭证密码。

2. 从 Registry 中拉取镜像

```
$ docker pull registry.cn-shanghai.aliyuncs.com/easonlin-devops-practice/action:[镜像版本号]
```

3. 将镜像推送到 Registry

```
$ docker login --username=tb1020550141 registry.cn-shanghai.aliyuncs.com
$ docker tag [ImageId] registry.cn-shanghai.aliyuncs.com/easonlin-devops-practice/action:[镜像版本号]
$ docker push registry.cn-shanghai.aliyuncs.com/easonlin-devops-practice/action:[镜像版本号]
```

请根据实际镜像信息替换示例中的 [ImageId] 和 [镜像版本号] 参数。

4. 选择合适的镜像仓库地址

从 ECS 推送镜像时，可以选择使用镜像仓库内网地址。推送速度将得到提升并且将不会损耗您的公网流量。

如果您使用的机器位于 VPC 网络，请使用 registry-vpc.cn-shanghai.aliyuncs.com 作为 Registry 的域名登录。

secrets.ACR_USERNAME: 阿里云容器服务registry的用户名。

secrets.ACR_PASSWORD: 阿里云容器服务registry的用户密码。

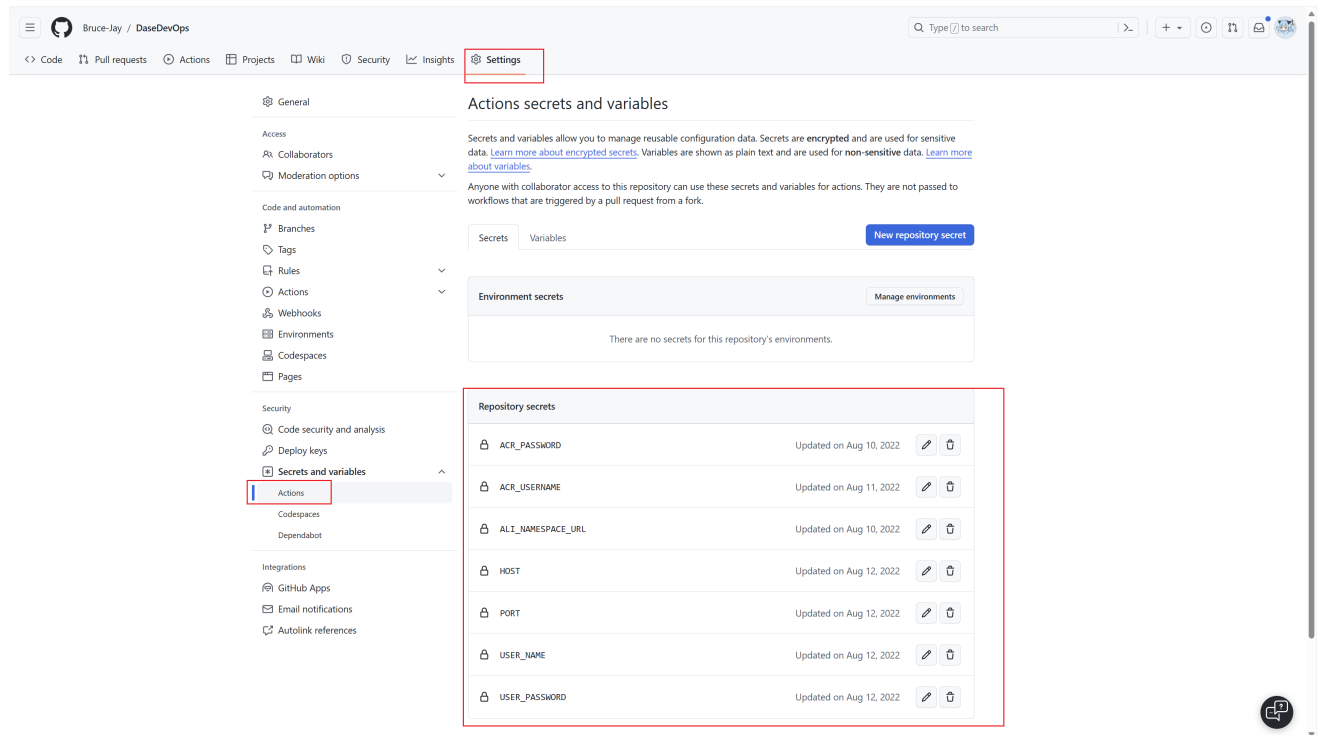
secrets.HOST: 服务器B的IP地址, 就是 ucloud 云主机的公有 ip

secrets.USER_NAME: 服务器B的登录用户名, 默认为 root

secrets.USER_PASSWORD: 服务器B的登录密码。

secrets.PORT: 服务器B的SSH开放端口, 默认是22。

这些 secrets都可以在代码仓库的“Setting →Secrets → Actions”界面进行设置, 使用自己账号的阿里云容器镜像服务中的参数。



新建 secret: 点击 New repository secret 按钮

Actions secrets and variables

Secrets and variables allow you to manage reusable configuration data. Secrets are **encrypted** and are used for sensitive data. [Learn more about encrypted secrets](#). Variables are shown as plain text and are used for **non-sensitive** data. [Learn more about variables](#).

Anyone with collaborator access to this repository can use these secrets and variables for actions. They are not passed to workflows that are triggered by a pull request from a fork.

Secrets

Variables

New repository secret

Environment secrets

Manage environments

There are no secrets for this repository's environments.

Repository secrets

输入 secret name 与 value 即可

Actions secrets / New secret

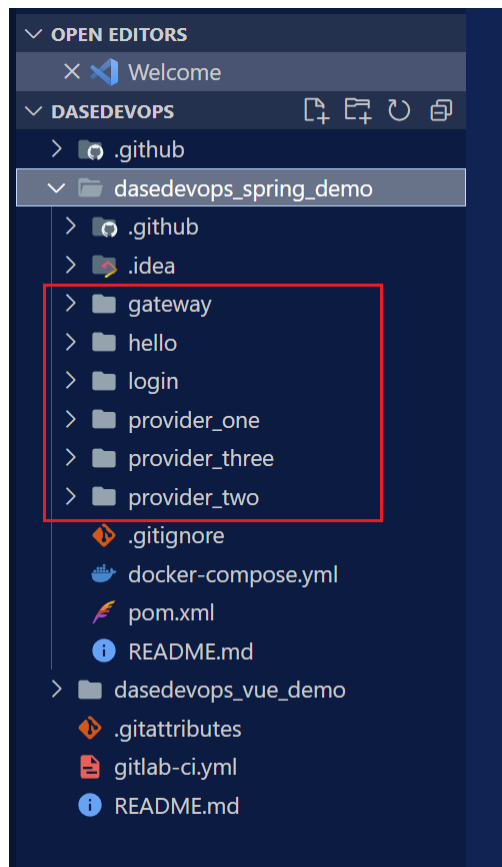
Name *

YOUR_SECRET_NAME

Secret *

Add secret

随后，进入微服务文件夹 dasedevops_spring_demo 下，进入以下六个文件夹的 Dockerfile 进行编辑。



注意到了吗，这六个文件夹就和 workflow 中的六个 MODULE 命名相同。

```
env:
  MODULE_1: gateway
  MODULE_2: hello
  MODULE_3: login
  MODULE_4: provider_one
  MODULE_5: provider_two
  MODULE_6: provider_three
```

编辑 DockerFile 也是为了分别将这六个 module 打包成镜像，上传到镜像仓库中。这么看，这个 yml 文件是不是也不那么复杂？🐼

gateway微服务的Dockerfile文件内容如下：

在六个文件夹下分别创建名为dockerfile的文件

```
FROM openjdk:8
MAINTAINER ningzhicheng
VOLUME /tmp
ADD ./target/*.jar /gateway.jar
ENTRYPOINT ["java","-jar","/gateway.jar"]
EXPOSE 8080
```

hello微服务的Dockerfile文件内容如下：


```
FROM openjdk:8
MAINTAINER ningzhicheng
VOLUME /tmp
ADD ./target/*.jar /hello.jar
ENTRYPOINT ["java","-jar","/hello.jar"]
EXPOSE 8001
```

login微服务的Dockerfile文件内容如下:

```
FROM openjdk:8
MAINTAINER ningzhicheng
VOLUME /tmp
ADD ./target/*.jar /login.jar
ENTRYPOINT ["java","-jar","/login.jar"]
EXPOSE 8000
```

provider_one微服务的Dockerfile文件内容如下:

```
FROM openjdk:8
MAINTAINER ningzhicheng
VOLUME /tmp
ADD ./target/*.jar /provider_one.jar
ENTRYPOINT ["java","-jar","/provider_one.jar"]
EXPOSE 8666
```

provider_two微服务的Dockerfile文件内容如下:

```
FROM openjdk:8
MAINTAINER ningzhicheng
VOLUME /tmp
ADD ./target/*.jar /provider_two.jar
ENTRYPOINT ["java","-jar","/provider_two.jar"]
EXPOSE 8667
```

provider_three微服务的Dockerfile文件内容如下:

```
FROM openjdk:8
MAINTAINER ningzhicheng
VOLUME /tmp
ADD ./target/*.jar /provider_three.jar
ENTRYPOINT ["java","-jar","/provider_three.jar"]
EXPOSE 8668
```

6.编写 docker-compose.yml 文件

在 dasedevops_spring_demo 下新建一个 docker-compose.yml 文件, 将以下内容复制到文件中

```
version: "3"

services:
```

```
providerOne:
  image: ${{ secrets.ALI_NAMESPACE_URL }}/provider_one:latest
  ports:
    - "8666:8666"

gateway:
  image: ${{ secrets.ALI_NAMESPACE_URL }}/gateway:latest
  ports:
    - "8080:8080"

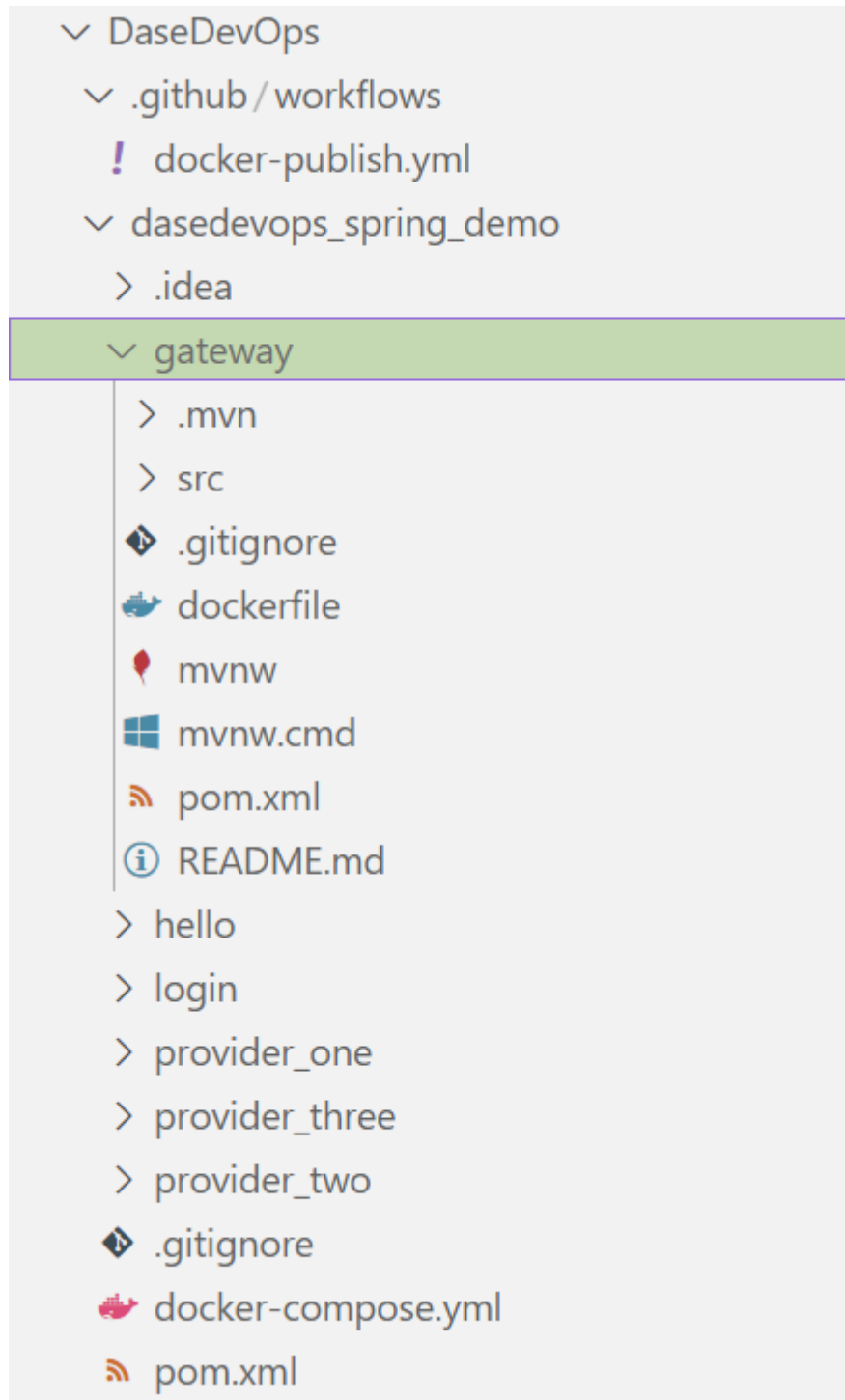
login:
  image: ${{ secrets.ALI_NAMESPACE_URL }}/login:latest
  ports:
    - "8000:8000"

hello:
  image: ${{ secrets.ALI_NAMESPACE_URL }}/hello:latest
  ports:
    - "8001:8001"

providerTwo:
  image: ${{ secrets.ALI_NAMESPACE_URL }}/provider_two:latest
  ports:
    - "8667:8667"

providerThree:
  image: ${{ secrets.ALI_NAMESPACE_URL }}/provider_three:latest
  ports:
    - "8668:8668"
```

文件目录如下：



随后将所有文件提交到暂存区，创建提交信息，推送提交。

```
git add .
git commit -m "test"
git push
```

查看自己仓库的 Actions 界面

☰

Bruce-Jay / DaseDevOps

🔍 Type [?] to search

<> Code

🔍 Pull requests

🔄 Actions

📁 Projects

📖 Wiki

🔒 Security

📊 Insights

⚙️ Settings

Actions

New workflow

All workflows

SpringCloud CI/CD with Docker

Management

Caches

Runners

Beta

SpringCloud CI/CD with Docker

docker-publish.yml

Filter workflow runs

...

18 workflow runs

Event

Status

Branch

Actor

test7

SpringCloud CI/CD with Docker #23: Commit 7ec0210 pushed by Bruce-Jay

master

13 minutes ago

2m 12s

...

test6

SpringCloud CI/CD with Docker #22: Commit 0e8d8c7 pushed by Bruce-Jay

master

17 minutes ago

19s

...

test5

SpringCloud CI/CD with Docker #21: Commit b6db0c50 pushed by Bruce-Jay

master

3 hours ago

3m 19s

...

test4

SpringCloud CI/CD with Docker #20: Commit 21c11df pushed by Bruce-Jay

master

3 hours ago

3m 15s

...

test3

SpringCloud CI/CD with Docker #19: Commit 6884165 pushed by Bruce-Jay

master

3 hours ago

1m 10s

...

test1

SpringCloud CI/CD with Docker #18: Commit bda9ece pushed by Bruce-Jay

master

3 hours ago

58s

...

test

SpringCloud CI/CD with Docker #17: Commit 1152fcd pushed by Bruce-Jay

master

20 hours ago

1m 8s

...

test

SpringCloud CI/CD with Docker #16: Commit 6a8b672 pushed by Bruce-Jay

master

yesterday

57s

...

add raw login

SpringCloud CI/CD with Docker #15: Commit b80bc4b pushed by Bruce-Jay

master

yesterday

49s

...

change java to openjdk

SpringCloud CI/CD with Docker #14: Commit 1df11da pushed by Bruce-Jay

master

yesterday

1m 4s

...

change openjdk to java

...

...

作业 2： 将成功的截图（示例如下）插入到实验报告中。

← SpringCloud CI/CD with Docker

test7 #23

Summary

Jobs

build

Docker Pull And Docker-compose...

Run details

Usage

Workflow file

Triggered via push 14 minutes ago

Status

Total duration

Artifacts

Bruce-Jay pushed 7ec0210 master

Success

2m 12s

—

docker-publish.yml

on: push

build 1m 46s

Docker Pull And Docker-co... 10s

Annotations

1 warning

build

The following actions uses node12 which is deprecated and will be forced to run on node16: actions/checkout@v2, actions/setup-jav

Show more

作业 3： 将阿里云容器镜像服务-个人实例-镜像仓库页面截图，插入实验报告。

示例如下图，必须包括这六个仓库。

阿里云

工作台

Q 搜索...

费用 ICP 备案 企业 支持 工单

📄 📁 📌 📧

🌐 全球

👤 用户头像

云服务器服务 / 实例列表 / 镜像仓库

← 个人实例

华东 2 (上海)

所在地域 华东

概览

仓库管理

镜像仓库

命名空间

代码源

访问凭证

创建镜像仓库

全部命名空间

Q 仓库名称

仓库名称	命名空间	仓库状态	仓库类型	仓库地址	创建时间	最近更新时间	操作
devops	easonlin-devops-practice	✓ 正常	私有	...	2022-07-16 15:37:03	2022-07-16 15:37:03	管理 删除
action	easonlin-devops-practice	✓ 正常	公开	...	2023-11-11 11:09:04	2023-11-11 11:09:05	管理 删除 置为私有
gateway	easonlin-devops-practice	✓ 正常	公开	...	2023-11-12 13:47:35	2023-11-12 16:43:05	管理 删除 置为私有
hello	easonlin-devops-practice	✓ 正常	公开	...	2023-11-12 13:55:00	2023-11-12 16:43:19	管理 删除 置为私有
login	easonlin-devops-practice	✓ 正常	公开	...	2023-11-12 13:55:18	2023-11-12 16:43:32	管理 删除 置为私有
provider_one	easonlin-devops-practice	✓ 正常	公开	...	2023-11-12 13:55:37	2023-11-12 16:43:44	管理 删除 置为私有
provider_two	easonlin-devops-practice	✓ 正常	公开	...	2023-11-12 13:55:56	2023-11-12 16:43:56	管理 删除 置为私有
provider_three	easonlin-devops-practice	✓ 正常	公开	...	2023-11-12 13:56:15	2023-11-12 16:44:08	管理 删除 置为私有

本次实验已结束，请同学们立即释放所有 uccloud 云主机资源！

作业请提交到坚果云 https://send2me.cn/OMVsV_xl/QLS56Ze4wBqJ4Q